

**Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Херсонский государственный педагогический университет»**

Институт информационных технологий

Кафедра программной инженерии и информационных технологий

ПРАКТИЧЕСКАЯ РАБОТА
по дисциплине:
«Программирование графических процессоров»
на тему:
«Практическая работа 1. Расширение языка CUDA C»

Выполнил:
Студент гр.: 12-25РПм
Трифонов Д.С.

Проверил
Преподаватель:
Алешов Е.В.

Оценка: _____
Дата: 01.10.2026

Херсон, 2026

ПРАКТИЧЕСКАЯ РАБОТА

Цель работы

Изучить расширения языка C/C++, которые используются при программировании на CUDA, и написать программу с функцией Hello World при разных спецификаторах и с ядром, увеличивающим элементы двумерного массива на два.

Контрольные вопросы

1. Перечислите расширения языка CUDA C.

В CUDA C язык C/C++ расширен пятью группами средств:

- спецификаторы функций: где функция выполняется и откуда её можно вызвать;
- спецификаторы переменных: в какой памяти GPU лежит переменная;
- встроенные векторные типы размерности 1–4 (`char1 ... double2`) и тип `dim3` для размера сетки и блока;
- встроенные переменные `threadIdx`, `blockIdx`, `blockDim` и `gridDim`: индексы нити и блока и размеры блока и сетки;
- директива запуска ядра `<<<gridDim, blockDim>>>`, которая задаёт иерархию нитей.

2. Как вычислить глобальный номер нити для трехмерного массива размера (DimX, DimY, DimZ)?

Сначала по каждому измерению получается индекс элемента, который обрабатывает данная нить. Он складывается из номера блока и номера нити внутри блока:

```
Idx = blockIdx.x * blockDim.x + threadIdx.x  
Idy = blockIdx.y * blockDim.y + threadIdx.y  
Idz = blockIdx.z * blockDim.z + threadIdx.z
```

Если массив хранится так, что измерение X меняется быстрее всего, линейный номер элемента равен:

```
global = Idx + DimX * (Idy + DimY * Idz)
```

Для двумерного массива из методички это та же формула без третьего измерения: `global = Idx + Idy * DimX`. В программе функция `globalIndex3D` считает трёхмерный номер, функция `globalIndex2D` — двумерный. Контрольный пример: `Dim = (4, 3, 2)`, `blockDim = (2, 2, 2)`, `blockIdx = (1, 0, 1)`,

`threadIdx = (1, 1, 0)`. Тогда `Idx = 3`, `Idy = 1`, `Idz = 2` и `global = 3 + 4 * (1 + 3 * 2) = 31`. Это значение печатает программа (рисунок 2).

3. Какие параметры задаются при вызове ядра?

При запуске ядра обязательно задаются размер сетки в блоках, размер блока в нитях и аргументы ядра:

```
kernel<<<gridDim, blockDim>>>(args);
```

`gridDim` и `blockDim` имеют тип `dim3`. Компоненты, которые не переданы в конструктор, равны единице, поэтому `dim3(2)` — это сетка (2, 1, 1), а `dim3(16, 16)` — блок (16, 16, 1). Дополнительно, третьим и четвёртым параметрами конфигурации, можно передать число байт динамической разделяемой памяти и поток CUDA:

```
kernel<<<gridDim, blockDim, sharedMem, stream>>>(args);
```

В программе использованы два запуска. Ядро Hello World: два блока по четыре нити, без аргументов. Ядро обработки массива: сетка `dim3(2, 2)`, блок `dim3(2, 2)`, аргументы — указатель на массив и его размеры. В файле `main.cu` вызов записан макросом `CUDA_LAUNCH`. Если файл компилирует `nvcc`, макрос раскрывается в обычную директиву `<<<grid, block>>>`.

4. Назовите спецификаторы переменных и функций в CUDA.

Спецификаторы функций:

Спецификатор	Где выполняется	Откуда вызывается
<code>__global__</code>	device (GPU)	host (CPU)
<code>__device__</code>	device (GPU)	device (GPU)
<code>__host__</code>	host (CPU)	host (CPU)

Спецификатор `__global__` обозначает ядро: функция возвращает `void`, а при вызове обязательно указывается конфигурация запуска. Спецификаторы `__host__` и `__device__` можно поставить вместе — тогда одна функция собирается и для CPU, и для GPU. Вместе `__global__` и `__host__` ставить нельзя. Если спецификатор не написан, функция считается `__host__`. Для функций устройства не допускаются рекурсия, статические переменные внутри функции и переменное число аргументов.

Спецификаторы переменных:

Спецификатор	Память GPU	Видимость	Время жизни
<code>__device__</code>	глобальная	вся сетка	вся программа
<code>__shared__</code>	разделяемая	один блок	время работы блока
<code>__constant__</code>	константная	вся сетка	вся программа

Такие переменные объявляются в одном файле и не помечаются как `extern`. Переменную `__shared__` нельзя инициализировать в месте объявления. Запись в `__constant__` с хоста выполняется функциями CUDA; в программе это `cudaMemcpyToSymbol`. Спецификаторы памяти не применяются к полям структуры.

Практическая часть

Задание 1. Hello World с разными спецификаторами

В `main.cu` сделаны четыре функции. `hostHello` помечена `__host__` и вызывается из `main`: она печатает строку с CPU. `deviceHello` помечена `__device__` и вызывается только из ядра: она печатает «Hello world» и номера блока и нити. `helloBoth` помечена одновременно `__host__` и `__device__`: один раз она вызывается из `main`, второй раз — из ядра, для нити (0, 0). `HelloWorld` помечена `__global__` и является ядром. Каждая его нить печатает строку вида «Hello world, номер блока, номер нити», как в примере из методички.

Ядро запущено как `HelloWorld<<<dim3(2), dim3(4)>>>()`: два блока, в каждом четыре нити, всего восемь строк. В выводе есть пары (0, 0) ... (0, 3) и (1, 0) ... (1, 3), то есть `blockIdx.x` и `threadIdx.x` различаются у разных нитей.

```
MINGW64: ~/Desktop/do-it/app-cuda

mitriy@DESKTOP-0NVCEI5 MINGW64 ~/Desktop/do-it/app-cuda
$ g++ -x c++ -std=c++17 -Wall -Wextra -static -o main.exe main.cu

mitriy@DESKTOP-0NVCEI5 MINGW64 ~/Desktop/do-it/app-cuda
$ ./main.exe
Практическая работа 1. Расширение языка CUDA C
Сборка: эмуляция иерархии нитей на CPU (на этом ПК нет CUDA-устройства)

===== 1. Hello World: спецификаторы функций =====
[__host__] Hello world
[__host__ __device__] Hello world (вызов с CPU), -1, -1

Запуск: HelloWorld<<<dim3(2), dim3(4)>>>()
[__device__] Hello world, 0, 0
[__host__ __device__] Hello world (вызов с GPU), 0, 0
Hello world, 0, 0
Hello world, 0, 1
Hello world, 0, 2
Hello world, 0, 3
Hello world, 1, 0
Hello world, 1, 1
Hello world, 1, 2
Hello world, 1, 3
```

Рисунок 1 — Сборка программы и выполнение задания 1

Задание 2. Ядро, увеличивающее элементы двумерного массива на два

Ядро `addTwo2D` получает указатель на массив и размеры `DimX` и `DimY`. Номер элемента считается по формулам методички:

```
idx = blockIdx.x * blockDim.x + threadIdx.x
idy = blockIdx.y * blockDim.y + threadIdx.y
global = idx + idy * DimX
```

Если `idx` не меньше `DimX` или `idy` не меньше `DimY`, нить выходит из ядра и ничего не записывает. Так отсекаются лишние нити, когда размер массива не делится на размер блока.

Число 2 хранится не в аргументе, а в константной переменной `c_delta`. С хоста оно записывается вызовом `cudaMemcpyToSymbol`. Внутри блока нить (0, 0) копирует `c_delta` в разделяемую переменную `delta`, после `__syncthreads()` каждая нить блока прибавляет `delta` к своему элементу. Глобальная переменная устройства `d_initialized` равна 1 и читается этой же нитью, чтобы спецификатор `__device__` тоже участвовал в работе ядра.

Проверялся массив 4×3 , элементы которого сначала равны 0, 1, ..., 11. Блок задан как `dim3(2, 2)`, сетка — `dim3(2, 2)`: нитей 16, элементов 12, четыре нити попадают за границу массива и пропускаются. После копирования результата обратно на хост каждый элемент стал больше исходного на 2. Программа сравнивает массивы и печатает «да».

В том же запуске на хосте показаны встроенные типы из теоретической части. `make_int2(2, 5)` даёт вектор с компонентами $x = 2$ и $y = 5$. Конструктор `dim3 block(16, 16)` заполняет не переданную компоненту z единицей, поэтому размер блока равен (16, 16, 1).

```
MINGW64: ~/Desktop/do-it/app-cuda (продолжение)
```

```
===== Встроенные векторные типы =====
int2 vec = make_int2(2, 5) -> (2, 5)
dim3 block(16, 16) -> (16, 16, 1)

===== Глобальный номер нити для массива 3D =====
Dim = (4, 3, 2), blockDim = (2, 2, 2)
blockIdx = (1, 0, 1), threadIdx = (1, 1, 0)
Idx = 3, Idy = 1, Idz = 2
global = Idx + DimX * (Idy + DimY * Idz) = 31

===== 2. Ядро: двумерный массив + 2 =====
Массив 4 x 3, блок dim3(2, 2), сетка dim3(2, 2)
Запуск: addTwo2D<<<dim3(2, 2), dim3(2, 2)>>>(d_data, 4, 3)
До:
    0    1    2    3
    4    5    6    7
    8    9   10   11
Ядро addTwo2D: d_initialized=1, delta=2, global=0
После (+2):
    2    3    4    5
    6    7    8    9
   10   11   12   13
Проверка: каждый элемент увеличен на 2 — да

mitriy@DESKTOP-0NVCEI5 MINGW64 ~/Desktop/do-it/app-cuda
$
```

Рисунок 2 — Векторные типы, глобальный индекс для массива 3D и результат ядра +2

Сборка на этом компьютере

Текст программы лежит в каталоге `app-cuda`, файл `main.cu`. По методичке программа на CUDA оформляется с расширением `.cu`, а не `.cpp`. Команда для машины, где установлен CUDA Toolkit:

```
nvcc -o main.exe main.cu
```

На компьютере, где получен вывод на рисунках 1 и 2, видеоадаптер — Intel HD Graphics 3000. CUDA-устройства нет, пакет CUDA Toolkit и компилятор `nvcc` не установлены, поэтому запустить ядро на GPU здесь нельзя. Тот же `main.cu` собирается компилятором `g++`. Если макрос `__CUDACC__` не определён (его определяет только `nvcc`), подключается файл `host_cuda_emu.h`: директива запуска выполняется циклом по блокам и нитям на CPU, а `cudaMalloc`, `cudaMemcpy` и `cudaMemcpyToSymbol` работают с обычной памятью процесса. Печать номеров нитей и проверка «каждый элемент увеличен на 2» при таком обходе совпадают с моделью выполнения CUDA. На компьютере с `nvcc` и GPU заголовок эмуляции в сборку не входит, ядра исполняются на устройстве.

Команда, которой собрана программа для рисунков:

```
g++ -x c++ -std=c++17 -Wall -Wextra -static -o main.exe main.cu
```

Ключ `-x c++` нужен потому, что у файла расширение `.cu`, а не `.cpp`. Предупреждений компилятора не было. Ожидание нажатия клавиши из примера методички (`getchar`) в программу не ставилось: иначе консоль не закрывается сама и вывод нельзя снять в пакетном запуске.

Листинг `main.cu`

```
// Практическая работа 1. Расширение языка CUDA C
//
// Сборка на машине с CUDA Toolkit:
//   nvcc -o main.exe main.cu
//
// Сборка на машине без nvcc (эмуляция сетки нитей на CPU):
//   g++ -x c++ -std=c++17 -Wall -Wextra -o main.exe main.cu

#include <stdio.h>
#include <stdlib.h>

#ifdef _WIN32
#ifndef NOMINMAX
#define NOMINMAX
#endif
#include <windows.h>
#endif

#if defined(__CUDACC__)
#include <cuda_runtime.h>
#include <device_launch_parameters.h>
#define CUDA_LAUNCH(fn, grid, block, ...) fn<<<grid, block>>>(__VA_ARGS__)
#else
```

```

#include "host_cuda_emu.h"
#endif

// --- спецификаторы переменных ---
__device__ int d_initialized = 1;
__constant__ float c_delta;

static void cudaCheck(cudaError_t err, const char *what) {
    if (err != cudaSuccess) {
        printf("Ошибка CUDA (%s): %s\n", what, cudaGetErrorString(err));
        exit(1);
    }
}

// Выполняется на CPU, вызывается с CPU.
__host__ void hostHello(void) {
    printf("[__host__] Hello world\n");
}

// Выполняется на GPU, вызывается из ядра.
__device__ void deviceHello(unsigned int block, unsigned int thread) {
    printf("[__device__] Hello world, %u, %u\n", block, thread);
}

// Один и тот же код доступен и на хосте, и на устройстве.
__host__ __device__ void helloBoth(const char *place, int block, int thread) {
    printf("[__host__ __device__] Hello world (%s), %d, %d\n", place, block, thread);
}

__host__ __device__ int globalIndex2D(int idx, int idy, int dimX) {
    return idx + idy * dimX;
}

__host__ __device__ int globalIndex3D(int idx, int idy, int idz, int dimX, int dimY) {
    return idx + dimX * (idy + dimY * idz);
}

// Ядро: учебный Hello world. Конфигурация задаётся при вызове.
__global__ void HelloWorld(void) {
    if (blockIdx.x == 0 && threadIdx.x == 0) {
        deviceHello(blockIdx.x, threadIdx.x);
        helloBoth("вызов с GPU", (int)blockIdx.x, (int)threadIdx.x);
    }
    printf("Hello world, %u, %u\n", blockIdx.x, threadIdx.x);
}

// Ядро задания 2: каждый элемент двумерного массива увеличивается на 2.
// Двойка лежит в константной памяти и раздаётся нитям блока через shared.
__global__ void addTwo2D(float *data, int dimX, int dimY) {
    __shared__ float delta;

    if (threadIdx.x == 0 && threadIdx.y == 0) {
        delta = c_delta;
    }
    __syncthreads();

    int idx = (int)(blockIdx.x * blockDim.x + threadIdx.x);
    int idy = (int)(blockIdx.y * blockDim.y + threadIdx.y);
    if (idx >= dimX || idy >= dimY) {
        return;
    }
}

```



```

    int global = globalIndex2D(idX, idY, dimX);
    data[global] = data[global] + delta;

    if (idX == 0 && idY == 0) {
        printf("Ядро addTwo2D: d_initialized=%d, delta=%.0f, global=%d\n",
            d_initialized, delta, global);
    }
}

static void printMatrix(const char *title, const float *data, int dimX, int dimY) {
    int y;
    int x;
    printf("%s\n", title);
    for (y = 0; y < dimY; ++y) {
        for (x = 0; x < dimX; ++x) {
            printf("%6.0f", data[y * dimX + x]);
        }
        printf("\n");
    }
}

int main(void) {
    const int dimX = 4;
    const int dimY = 3;
    const int blockX = 2;
    const int blockY = 2;
    const int n = dimX * dimY;
    const int gridX = (dimX + blockX - 1) / blockX;
    const int gridY = (dimY + blockY - 1) / blockY;

    float *h_in = NULL;
    float *h_out = NULL;
    float *d_data = NULL;
    float delta = 2.0f;
    int i;
    int ok;
    int2 vec;
    dim3 sampleBlock(16, 16);

    int bx = 1, by = 0, bz = 1;
    int tx = 1, ty = 1, tz = 0;
    int bdx = 2, bdy = 2, bdz = 2;
    int exDimX = 4, exDimY = 3, exDimZ = 2;
    int idX, idY, idZ, global3;

#ifdef _WIN32
    SetConsoleOutputCP(65001);
    SetConsoleCP(65001);
#endif

    printf("Практическая работа 1. Расширение языка CUDA C\n");
    #if defined(__CUDACC__)
        printf("Сборка: nvcc (ядра выполняются на GPU)\n\n");
    #else
        printf("Сборка: эмуляция иерархии нитей на CPU "
            "(на этом ПК нет CUDA-устройства)\n\n");
    #endif

    printf("===== 1. Hello World: спецификаторы функций =====\n");
    hostHello();
    helloBoth("вызов с CPU", -1, -1);
    printf("\nЗапуск: HelloWorld<<<dim3(2), dim3(4)>>>()\n");
}

```

```

CUDA_LAUNCH(HelloWorld, dim3(2), dim3(4));
cudaCheck(cudaGetLastError(), "HelloWorld");
cudaCheck(cudaDeviceSynchronize(), "HelloWorld sync");

printf("\n===== Встроенные векторные типы =====\n");
vec = make_int2(2, 5);
printf("int2 vec = make_int2(2, 5) -> (%d, %d)\n", vec.x, vec.y);
printf("dim3 block(16, 16) -> (%u, %u, %u)\n",
    sampleBlock.x, sampleBlock.y, sampleBlock.z);

printf("\n===== Глобальный номер нити для массива 3D =====\n");
printf("Dim = (%d, %d, %d), blockDim = (%d, %d, %d)\n",
    exDimX, exDimY, exDimZ, bdx, bdy, bdz);
printf("blockIdx = (%d, %d, %d), threadIdx = (%d, %d, %d)\n",
    bx, by, bz, tx, ty, tz);
idx = bx * bdx + tx;
idy = by * bdy + ty;
idz = bz * bdz + tz;
global3 = globalIndex3D(idx, idy, idz, exDimX, exDimY);
printf("Idx = %d, Idy = %d, Idz = %d\n", idx, idy, idz);
printf("global = Idx + DimX * (Idy + DimY * Idz) = %d\n", global3);

printf("\n===== 2. Ядро: двумерный массив + 2 =====\n");
printf("Массив %d x %d, блок dim3(%d, %d), сетка dim3(%d, %d)\n",
    dimX, dimY, blockX, blockY, gridX, gridY);
printf("Занеск: addTwo2D<<<dim3(%d, %d), dim3(%d, %d)>>>(d_data, %d, %d)\n",
    gridX, gridY, blockX, blockY, dimX, dimY);

h_in = (float *)malloc((size_t)n * sizeof(float));
h_out = (float *)malloc((size_t)n * sizeof(float));
if (h_in == NULL || h_out == NULL) {
    printf("Не удалось выделить память на хосте\n");
    return 1;
}

for (i = 0; i < n; ++i) {
    h_in[i] = (float)i;
}

printMatrix("До:", h_in, dimX, dimY);

cudaCheck(cudaMalloc((void **)&d_data, (size_t)n * sizeof(float)), "cudaMalloc");
cudaCheck(cudaMemcpy(d_data, h_in, (size_t)n * sizeof(float),
    cudaMemcpyHostToDevice), "HtoD");
cudaCheck(cudaMemcpyToSymbol(c_delta, &delta, sizeof(delta)), "cudaMemcpyToSymbol");

CUDA_LAUNCH(addTwo2D, dim3(gridX, gridY), dim3(blockX, blockY), d_data, dimX, dimY);
cudaCheck(cudaGetLastError(), "addTwo2D");
cudaCheck(cudaDeviceSynchronize(), "addTwo2D sync");
cudaCheck(cudaMemcpy(h_out, d_data, (size_t)n * sizeof(float),
    cudaMemcpyDeviceToHost), "DtoH");

printMatrix("После (+2):", h_out, dimX, dimY);

ok = 1;
for (i = 0; i < n; ++i) {
    if (h_out[i] != h_in[i] + 2.0f) {
        ok = 0;
    }
}

printf("Проверка: каждый элемент увеличен на 2 - %s\n", ok ? "да" : "нет");

```

```
    cudaFree(d_data);  
    free(h_in);  
    free(h_out);  
    return ok ? 0 : 1;  
}
```

Вывод

В работе перечислены расширения CUDA C, записаны формулы глобального номера нити для двумерного и трёхмерного массивов и параметры конфигурации запуска ядра. В программе реализованы функции Hello World со спецификаторами `__host__`, `__device__`, `__host__ __device__` и `__global__`, а также ядро `addTwo2D`, которое прибавляет 2 к каждому элементу двумерного массива. Запуск показал разные пары (`blockIdx.x`, `threadIdx.x`) и подтвердил, что все 12 элементов массива 4×3 увеличились на 2.